MINI PROJECT REPORT

ON

"Office Resource Management System"

SUBMITTED IN PARTIAL FULLFILLMENT OF THE REQUIREMENTS OF DEGREE OF

BACHELOR OF ENGINEERING

BY

Arvind Patil (T2, 2401079)

Harsh Phale (T2, 2401084)

Arya Rane (T2, 2401090)

SUBJECT

AGILE SOFTWARE DEVELOPMENT AND DEVOPS

SUPERVISOR

Prof. Sanober Shaikh



DEPARTMENT OF ARTIFICIAL INTELLIGENCE & DATA SCIENCE
THADOMAL SHAHANI ENGINEERING COLLEGE, ADVOCATE NARI GURSAHANI MARG, 37TH RD, OFF LINKING ROAD, TPS III, BANDRA (WEST), MUMBAI, 400050
UNIVERSITY OF MUMBAI
[2025-26]

THADOMAL SHAHANI ENGINEERING COLLEGE, ADVOCATE NARI GURSAHANI MARG, 37TH RD, OFF LINKING ROAD, TPS III, BANDRA (WEST), MUMBAI, 400050 A.Y. - 2025-26

Certificate

This is to certify that the Mini Project entitled **"Office Resource Management System"** is a bonafide work of **Arvind Patil, Harsh Phale, Arya Rane** submitted to the University of Mumbai in partial fulfillment of the requirement for the award of the degree of **"Undergraduate"** in **"Artificial Intelligence and Data Science"**.

_____
Prof. Sanober Shaikh
(Supervisor)

Abstract

The Office Resource Management System (ORMS) is a web application that helps a company manage its office assets (laptops, monitors, printers) and shared spaces (meeting rooms, projectors). Employees can request equipment, book rooms and report faulty items, while admins approve requests, avoid double bookings and track repairs. It is built with React, Node.js, Express and MongoDB with two roles: Admin and Employee.

The project also shows a complete Agile and DevOps cycle: planning in Jira (Epics, Stories, 3 Sprints), code on GitHub, automatic testing with GitHub Actions, packaging with Docker, deployment on Kubernetes, and live monitoring with Prometheus and Grafana.

TABLE OF CONTENTS

Content **Page No.**

Abstract **3**

1. Introduction **5**

1.1 Background 5

1.2 Problem Statement 5

2. Implementation **6**

2.1 Technologies Used 6

2.2 Code Structure 6

3. Results and Discussion **8**

3.1 Result 8

3.2 Discussion 12

4. Conclusion **9**

4.1 Conclusion 9

5. References **9**

1. Introduction

1.1 Background

Offices own many assets such as laptops, monitors and projectors, and shared spaces such as meeting rooms. Many companies still track them in Excel sheets or messages, which causes confusion about who has which item and leads to double bookings and repeated issue of broken equipment.

ORMS brings all of this into one web application with two roles, ADMIN and EMPLOYEE. As this project is for Agile Software Development and DevOps, we also used a modern delivery process: Jira for Agile planning, Git/GitHub for code, GitHub Actions for CI, Docker for packaging, Kubernetes for deployment, and Prometheus with Grafana for monitoring.

1.2 Problem Statement

Manual handling of office resources has these problems:

- No clear record of which asset is available or who holds it.
- Same asset can be given twice and the same room can be booked twice.
- No proper approval, return date or history for requests.
- Broken items are not reported and get issued again.

Problem statement: To build a secure, role-based web application for managing office assets and room bookings with approval, conflict prevention and maintenance tracking, and to deliver it using Agile and DevOps practices.

2. Implementation

2.1 Technologies Used

Technology	Use in the Project
React + Vite + Tailwind	Frontend user interface and production build
Node.js 22 + Express	Backend REST API
MongoDB Atlas + Mongoose	Cloud database and data models
JWT	Secure login and role checks
Jira Software	Agile planning: Epics, Stories, Story Points, 3 Sprints
Git & GitHub	Version control with main, develop and feature branches
GitHub Actions	CI: runs tests, frontend build and Docker image build on every push
Docker	Packages frontend and backend; multi-stage build cuts frontend image from ~300 MB to ~25 MB
Docker Compose	Runs the full app locally with one command
Kubernetes	2 pods each for frontend and backend, health probes, Ingress, auto-scaling (2 to 5 pods at 70% CPU)
Prometheus	Collects backend metrics from /api/metrics every 5 seconds
Grafana	Dashboard showing requests, latency, errors and memory

2.2 Code Structure

```
office-resource-management/
├── .github/workflows/ci.yml    # GitHub Actions CI
├── backend/                    # Express API: controllers, models, routes,
│   │                           # middleware, services, metrics.js
│   ├── tests/api.test.js       # automated tests
│   └── Dockerfile
├── frontend/                   # React app + multi-stage Dockerfile + nginx.conf
├── k8s/                        # deployments, services, ingress, HPA,
│                               # prometheus, grafana, kustomization
├── docker-compose.yml └── README.md
```

Main Features and Flow

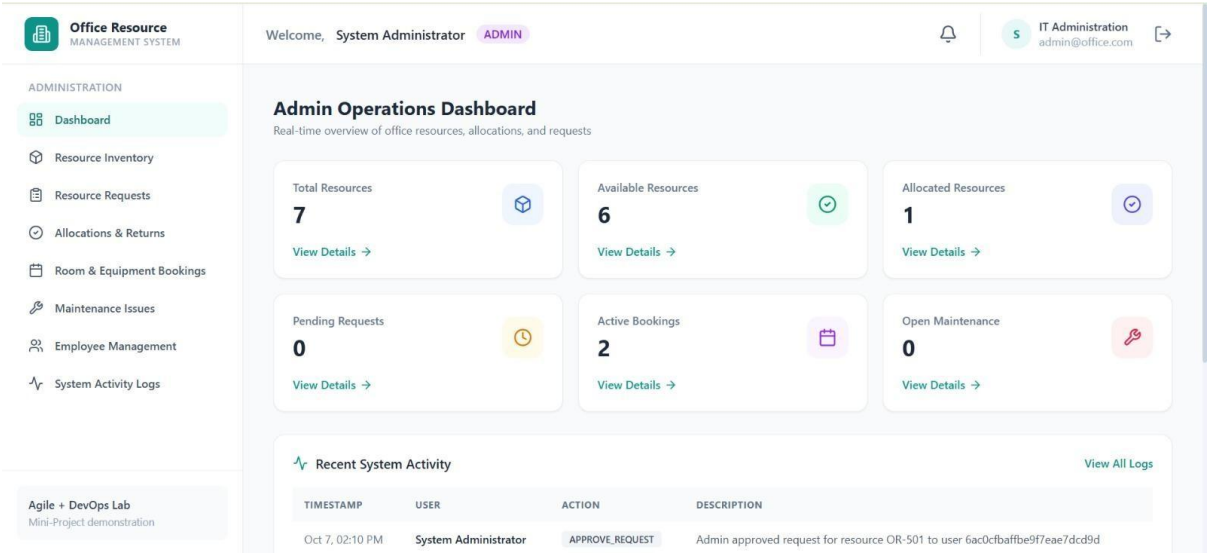
- **Role-based access:** Signup creates only EMPLOYEE accounts; admins are seeded.
- **Requests and bookings:** Admin approves or rejects requests; one asset cannot be allotted twice. Overlapping room bookings give 409 Conflict (newStart < existingEnd AND newEnd > existingStart).
- **Maintenance and logs:** Reported defects mark the asset UNDER_MAINTENANCE; admin has dashboards, notifications and an activity log.
- **DevOps flow:** Jira plan → GitHub branch → GitHub Actions (tests, builds) → merge → kubectl apply -k k8s → Grafana monitoring.

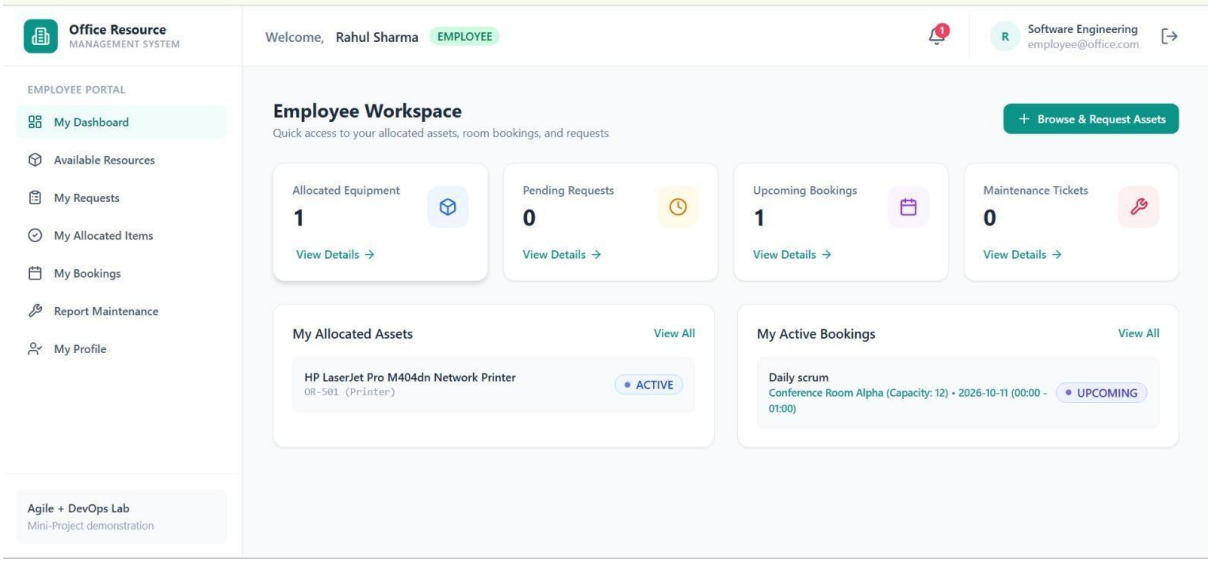
2.3 Team Contribution

Member	Role
Arvind Patil	DevOps, backend architecture and pipeline lead
Harsh Phale	Business logic, allocation and workflow validation (QA)
Arya Rane	Frontend architecture, UI/UX and React components

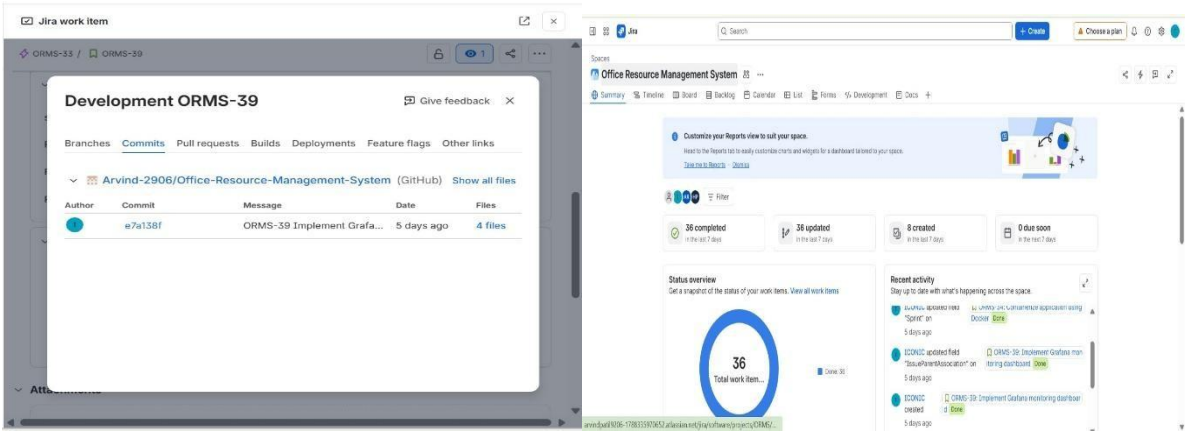
3. Results and Discussion

3.1 Result

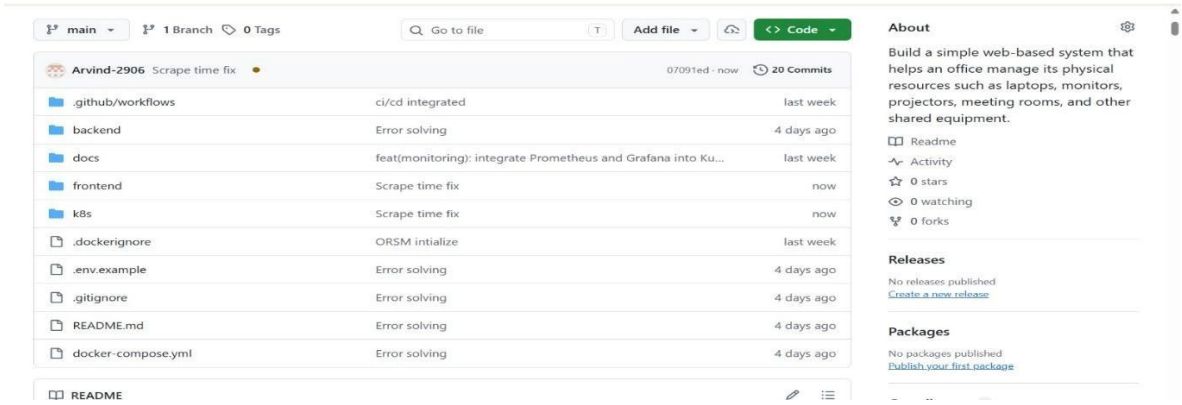




JIRA:

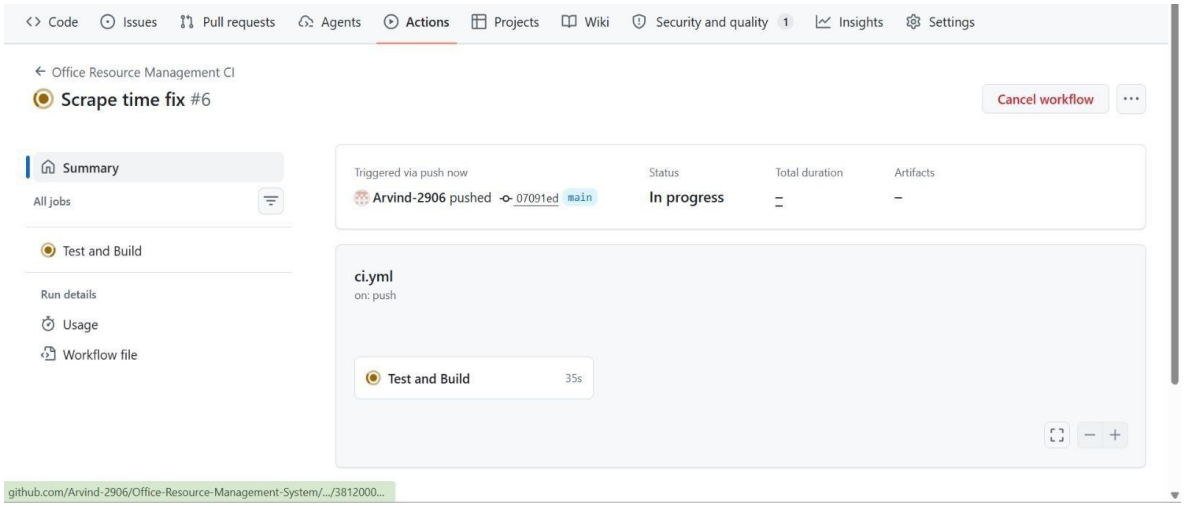


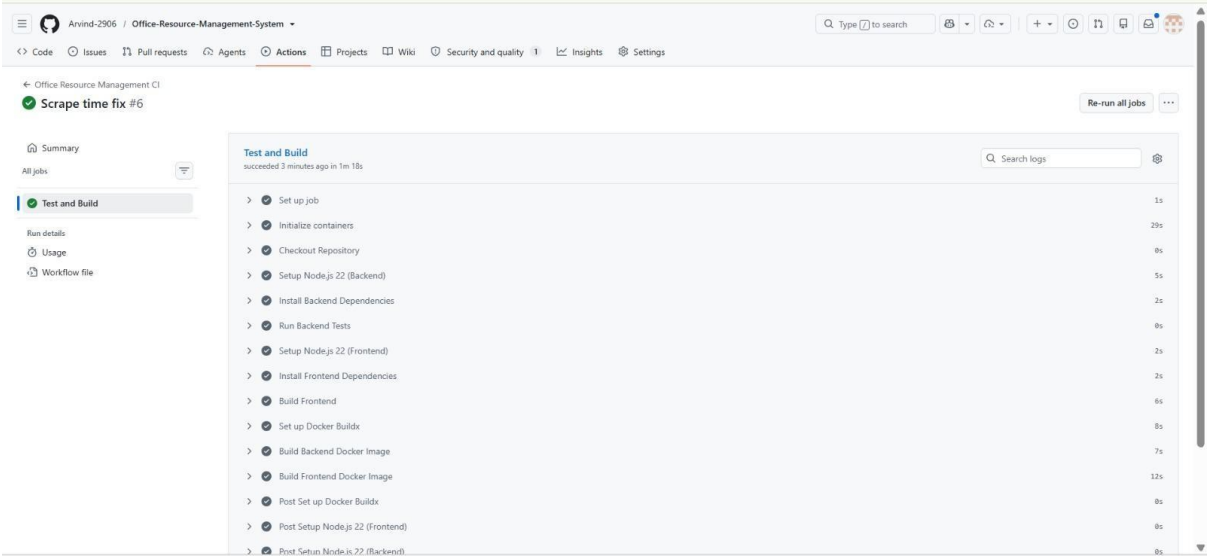
GIT HUB:



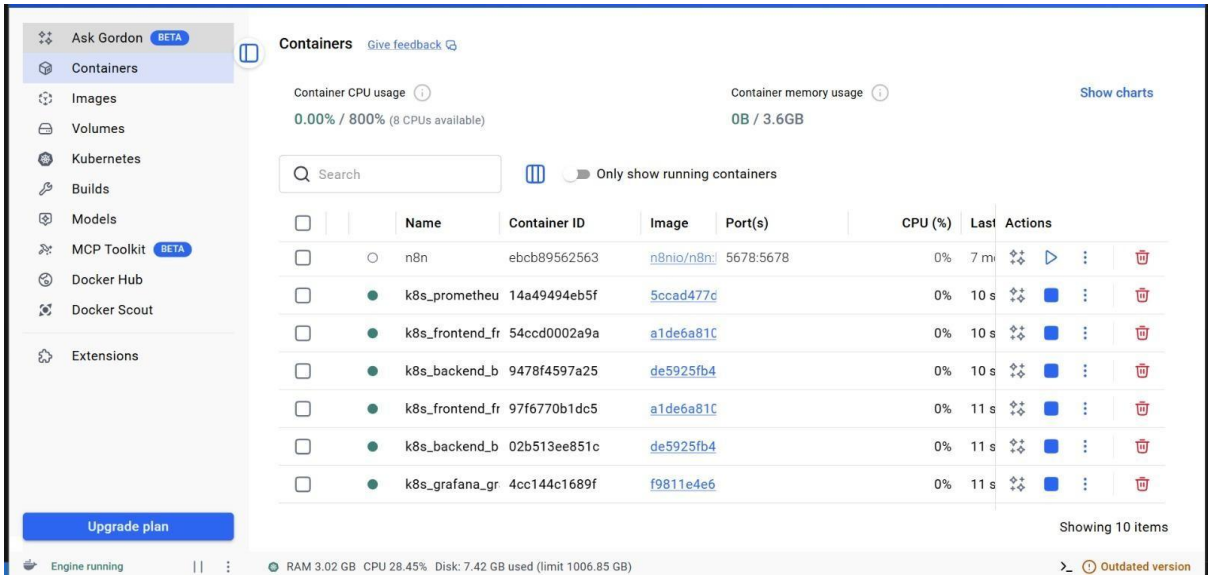

```
PS C:\Arvind\ASDd> git add .
warning: in the working copy of 'frontend/src/pages/auth/Login.jsx', LF will be re
placed by CRLF the next time Git touches it
PS C:\Arvind\ASDd> git commit -m "Scrape time fix"
[main 07091ed] Scrape time fix
 2 files changed, 3 insertions(+), 3 deletions(-)
PS C:\Arvind\ASDd> git push origin main
Enumerating objects: 17, done.
Counting objects: 100% (17/17), done.
Delta compression using up to 8 threads
Compressing objects: 100% (9/9), done.
Writing objects: 100% (9/9), 740 bytes | 740.00 KiB/s, done.
Total 9 (delta 6), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (6/6), completed with 6 local objects.
To https://github.com/Arvind-2906/Office-Resource-Management-System.git
   1016651..07091ed  main -> main
```

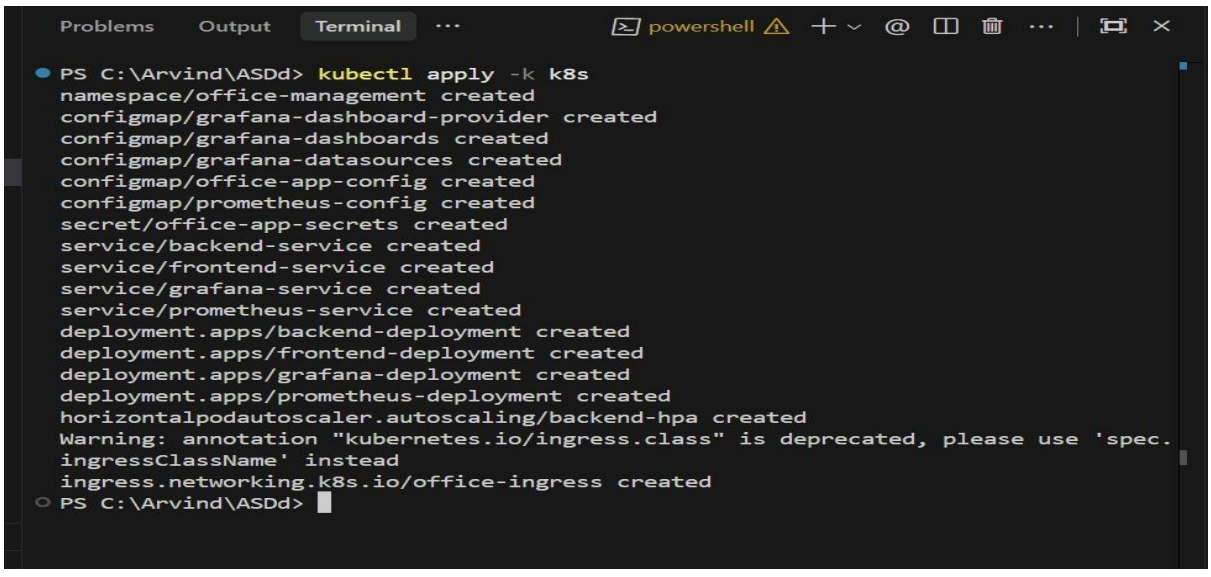
GIT HUB ACTIONS:



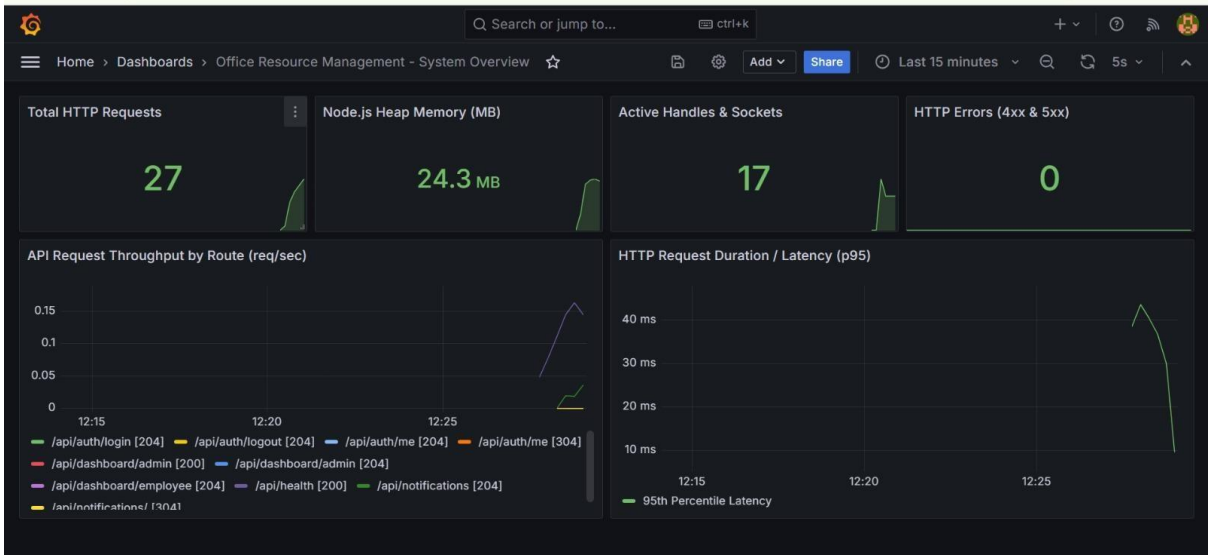


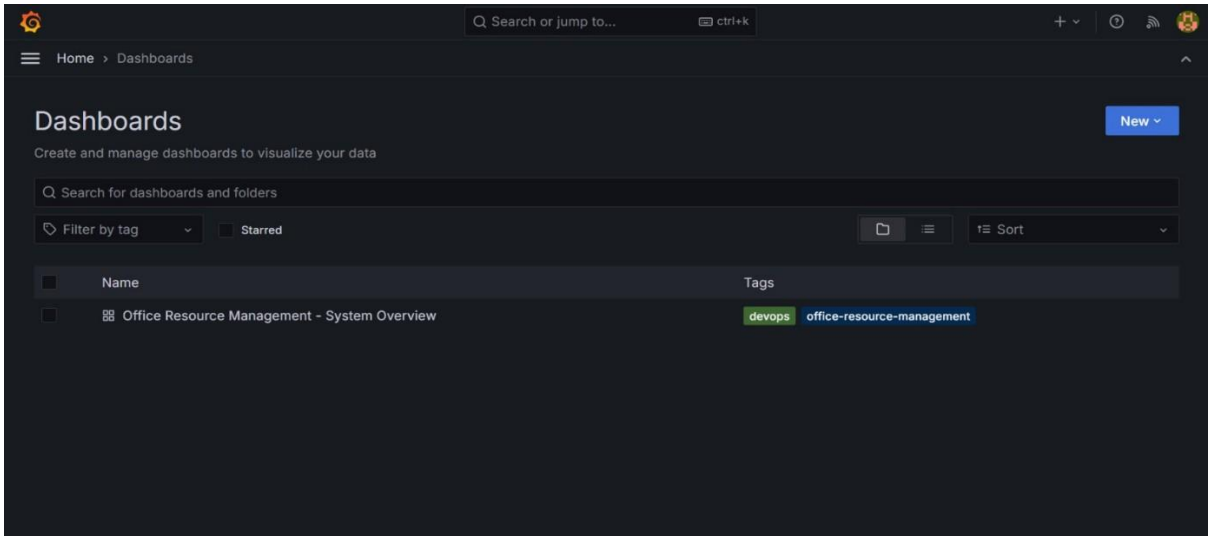
DOCKER/KUBERNETES:



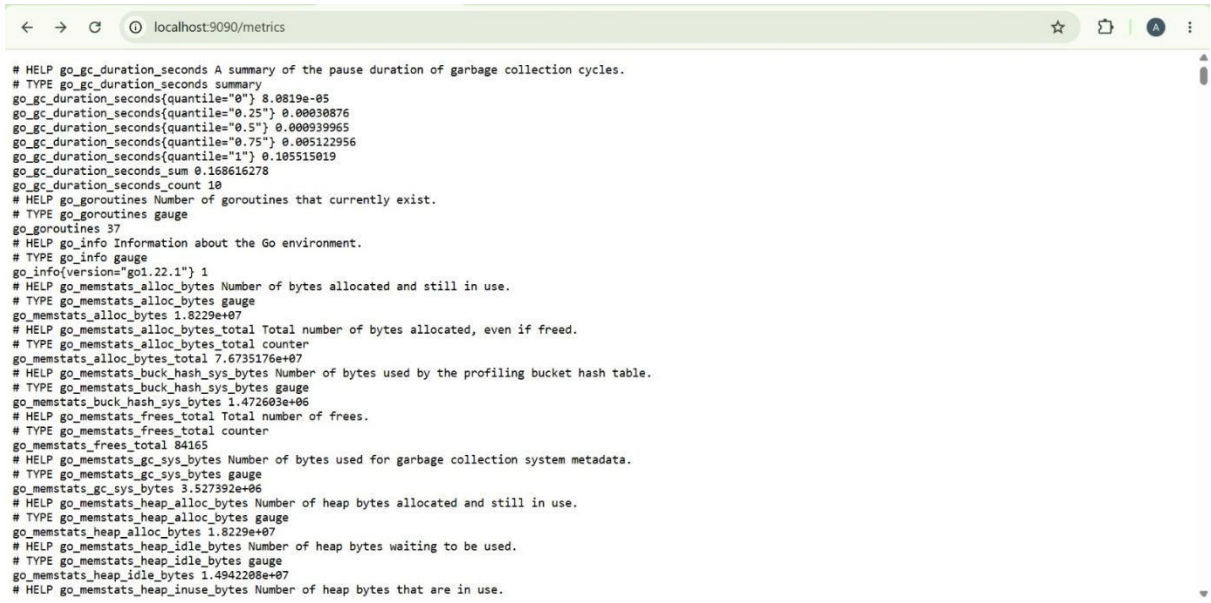


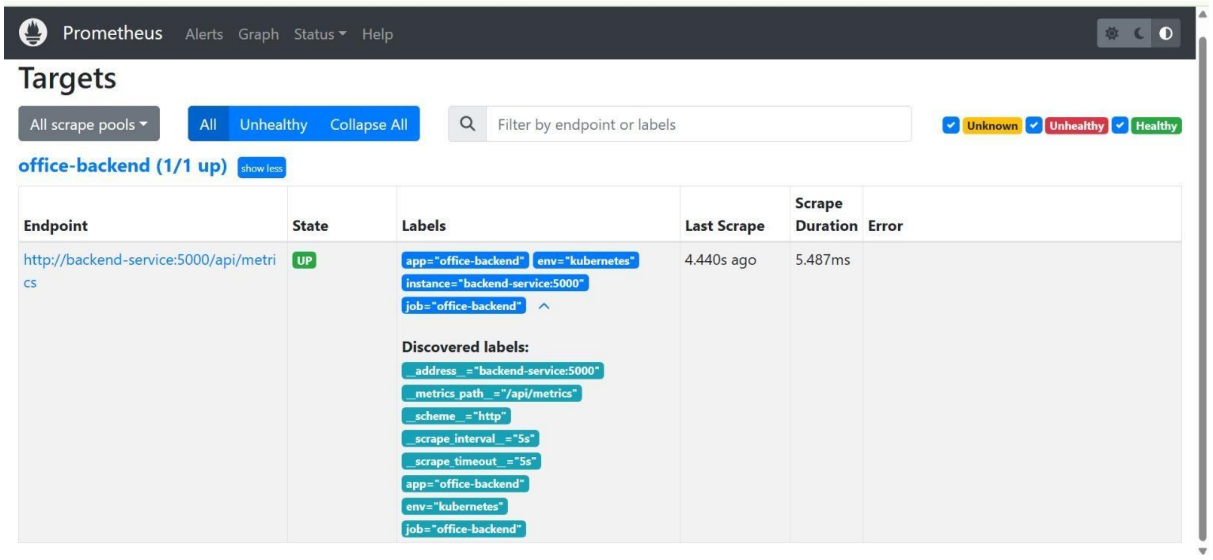
GRAFANA:





PROMETHEUS:





3.2 Discussion

- Tests confirm that signup cannot create an Admin, employees get 403 on admin actions, overlapping bookings give 409, adjacent bookings are accepted and an asset cannot be allotted twice.
- Docker multi-stage build reduced the frontend image from about 300 MB to about 25 MB.
- Kubernetes keeps the app running with 2 replicas and health probes, and can scale the backend up to 5 pods.
- GitHub Actions stops broken code from being merged because the build fails on any error.
- Grafana gives live view of traffic, response time, errors and memory.
- Limitation: deployment to Kubernetes is manual because the cluster runs on a local laptop.

Key Observations

- **Docker image size:** The multi-stage build reduced the frontend image from about 300 MB to about 25 MB, which makes it faster to download and safer because there is no Node.js in the final image.
- **Reliability:** With 2 replicas for each service and health probes, Kubernetes keeps the app running even if one pod crashes, and updates are done without downtime.
- **Scalability:** The HPA can scale the backend from 2 to 5 pods when CPU usage goes above 70%.

- **Quality:** The CI pipeline blocks broken code from reaching the main branch because the build turns red when a test or build fails.
- **Visibility:** Grafana shows request count, speed, errors and memory live, so problems like slow routes or memory leaks can be seen quickly.
- **Security:** Passwords and keys are kept in Kubernetes Secrets and ignored in Git (.env and k8s/secret.yaml), and only .example files are committed.

Future Scope

- Automatic deployment (CD) to a cloud Kubernetes cluster.
- Email or mobile notifications for approvals and return reminders.
- QR code or barcode scanning for assets.
- Usage reports and analytics for better planning of purchases. Alerts in Grafana or Prometheus for high errors or slow responses

4. Conclusion

4.1 Conclusion

The Office Resource Management System successfully solves the common office problems of asset tracking, equipment approval, room booking clashes and repair follow-up. It provides separate and secure access for Admins and Employees, prevents double allocation and double booking, and keeps a complete record of all activities.

More importantly, the project shows a complete Agile and DevOps cycle. Jira helped plan and track the work in sprints. Git and GitHub managed the code and teamwork. GitHub Actions tested and verified every change automatically. Docker packaged the app in small, portable images and Docker Compose made local running simple. Kubernetes made the app reliable and scalable, and Prometheus with Grafana gave live monitoring of the running system.

Through this mini project, the team learned how a real software product moves from an idea to a running and monitored application, and how teamwork, automation and monitoring improve the quality and speed of software delivery.

5. References

[1] GitHub Repository: https://github.com/Arvind-2906/Office-Resource-Management-System

[2] Docker: https://docs.docker.com

[3] Kubernetes: https://kubernetes.io/docs

[4] GitHub Actions: https://docs.github.com/actions

[5] Prometheus: https://prometheus.io/docs

[6] Grafana: https://grafana.com/docs

[7] Jira Software: https://support.atlassian.com/jira-software-cloud

[8] React: https://react.dev | Express: https://expressjs.com | MongoDB Atlas: https://www.mongodb.com/docs/atlas